

A Hidden Markov Model Part-of-Speech Tagger with Laplace Smoothing and Log-Space Viterbi Decoding on the Brown Corpus

Udomveasna CHHEANG

*dept. Applied Mathematics and Statistics
Institute of Technology of Cambodia
Phnom Penh, Cambodia
udomveasnachheang@gmail.com*

Lyheng SIENG

*dept. Applied Mathematics and Statistics
Institute of Technology of Cambodia
Phnom Penh, Cambodia
lyhengkp900@gmail.com*

Ratana SEING

*dept. Applied Mathematics and Statistics
Institute of Technology of Cambodia
Phnom Penh, Cambodia
ratana.seing@outlook.com*

Denis HEANG

*dept. Applied Mathematics and Statistics
Institute of Technology of Cambodia
Phnom Penh, Cambodia
heangdenis011468@gmail.com*

Abstract—Part-of-speech (POS) tagging is a foundational task in natural language processing in which each word in a sentence is assigned its correct grammatical category. The task is difficult because many words are syntactically ambiguous: a word such as *watch* may act as a noun or a verb depending on context. This paper presents a Hidden Markov Model (HMM) POS tagger implemented entirely from first principles and trained and evaluated on the Brown Corpus using the twelve-tag universal tagset. The model parameters, the initial, transition and emission distributions are estimated by maximum-likelihood frequency counting on a randomly shuffled 80% training partition, using a fixed random seed so that the split and all reported figures are reproducible. Two practical obstacles that arise when moving from textbook theory to a corpus of real text are addressed directly. First, out-of-vocabulary words are handled with add- α (Laplace) smoothing of the emission matrix together with a dedicated unseen-word probability mass, preventing the zero probabilities that would otherwise make unseen words untaggable. Second, the multiplicative Viterbi recursion is reformulated in log-space to eliminate the floating-point underflow that destroys long-sentence decoding. On the held-out 20% test partition the textbook add-one ($\alpha = 1$) setting attains only 93.80%, because it substantially over-smooths a vocabulary of this size. The smoothing strength is therefore chosen by a grid search on a validation set carved from the training data rather than on the test set; this selects $\alpha = 10^{-9}$ as optimal and raises word-level accuracy to 96.05% on the test partition. Because the test data play no part in selecting α , this figure is an unbiased estimate. A detailed error analysis of failed sentences shows that the residual errors are dominated by rare or unseen content words and by syntactic dependencies that exceed the reach of the first-order Markov assumption, clarifying both the strengths and the structural limits of HMMs for sequence labeling.

Index Terms—Part-of-Speech Tagging, Hidden Markov Model, Viterbi Algorithm, Laplace Smoothing, Log-Space

Decoding, Out-of-Vocabulary Words, Brown Corpus, Sequence Labeling, Natural Language Processing.

I. INTRODUCTION

Assigning a part-of-speech (POS) tag such as noun, verb, adjective, determiner and so on to every word in a sentence is one of the oldest and most widely used components of a natural-language-processing pipeline. Reliable POS tags support downstream tasks such as syntactic parsing, named-entity recognition, information extraction and machine translation. The central difficulty of the task is lexical ambiguity: a single surface form frequently corresponds to several grammatical categories, and only the surrounding context disambiguates it. The word *watch*, for instance, is a noun in “my watch is broken” but a verb in “watch the dog”; only the words around it reveal which reading is intended.

Resolving this ambiguity therefore requires a model that reasons about whole sequences rather than isolated words. The Hidden Markov Model (HMM) is the classical probabilistic answer. It treats the observed words as emissions of an underlying, unobserved sequence of grammatical states and selects the tag sequence that is jointly most probable given the words. Although more recent neural taggers achieve higher accuracy, the HMM remains the canonical vehicle for understanding sequence labeling because every one of its parameters has a transparent statistical meaning and the optimal decoding is computed exactly by dynamic programming.

This paper documents the construction of such a tagger from scratch and its training and evaluation on the Brown Corpus. The contribution is not a new algorithm but a careful, reproducible engineering account of the two problems that separate the textbook HMM from one that actually runs on a large corpus of real sentences. The first problem is the out-of-vocabulary (OOV) word: the test set inevitably contains words never observed during training, and an unsmoothed emission model assigns them zero probability under every tag, which makes the entire sentence containing them

undecodable. The second problem is numerical underflow: the Viterbi algorithm multiplies many probabilities, each strictly below one, so for sentences of even moderate length the running product collapses to zero in floating-point arithmetic long before decoding finishes.

To resolve these problems the emission distribution is smoothed with the add- α (Laplace) estimator and given an explicit reserve of probability for unseen words, and the Viterbi recursion is transformed into log-space so that the fragile product of probabilities becomes a numerically stable sum of log-probabilities. The resulting system is evaluated by word-level accuracy on a held-out portion of the corpus, and its mistakes are examined in detail.

The specific objectives of this work are as follows:

- Formulate POS tagging as decoding in a first-order Hidden Markov Model and estimate its parameters from the Brown Corpus by maximum-likelihood counting.
- Apply add- α Laplace smoothing to the emission matrix and reserve an explicit probability mass for out-of-vocabulary words so that unseen words remain taggable, and select the smoothing strength α by grid search on a held-out validation set rather than on the test data.
- Re-derive and implement the Viterbi decoder in log-space to eliminate floating-point underflow on long sentences.
- Evaluate the tagger by word-level accuracy on a held-out test partition and analyse its errors to expose the limits of the first-order Markov assumption.

The remainder of the paper is organised as follows. Section II reviews background on POS tagging, HMMs, smoothing and the Viterbi algorithm. Section III states the HMM formulation. Section IV details the methodology: the dataset and its partitioning, parameter estimation, Laplace smoothing and the log-space decoder. Section V describes the experimental setup and evaluation metric. Section VI reports the accuracy, presents the error analysis and discusses the limitations of the model. Section VII concludes.

II. BACKGROUND AND RELATED WORK

A. Part-of-Speech Tagging and Syntactic Ambiguity

Early POS taggers were rule-based, applying hand-written contextual rules to disambiguate words. Such systems are interpretable but laborious to build and brittle on unanticipated input. Data-driven stochastic taggers, which learn disambiguation from a tagged corpus, became dominant once large annotated corpora became available. Church [7] demonstrated an early stochastic parts program for unrestricted text, and Brants [8] later showed the TnT tagger that a carefully smoothed statistical model is highly competitive, with the HMM tagger as the prototypical member of this family. The training data for the present work is the Brown Corpus of Francis and Kučera [5], accessed through the Natural Language Toolkit of Bird, Klein and

Loper [4]. Because the number of distinct grammatical categories depends on the tagset, this work uses the coarse twelve-category universal tagset of Petrov, Das and McDonald [6], which collapses the fine-grained Brown tags into a compact, language-independent inventory and is well suited to a from-scratch implementation.

B. Hidden Markov Models for Sequence Labeling

A Hidden Markov Model describes a sequence of observations as being generated by a sequence of hidden states under two simplifying assumptions. As set out in the classic tutorial of Rabiner [1] and in the standard treatments of Jurafsky and Martin [2] and Manning and Schütze [9], the first-order Markov assumption states that the probability of a state depends only on the immediately preceding state, while the output-independence assumption states that each observation depends only on the state that emits it. For POS tagging the hidden states are the grammatical tags and the observations are the words. These assumptions reduce the number of parameters to a tractable level and make exact decoding possible, but, as the error analysis later shows, they also bound the linguistic phenomena the model can capture.

C. Smoothing for Unseen Events

Maximum-likelihood estimation assigns zero probability to any word-tag pairing not seen in training. Because a test corpus always contains unseen words, an unsmoothed emission model is unusable in practice. Smoothing redistributes a small amount of probability mass to unseen events. Add- α (Laplace) smoothing, the simplest such method, adds a constant pseudo-count α to every count before normalising. As the comparative study of Chen and Goodman [10] documents, more sophisticated discounting schemes generally outperform add- α smoothing on language-modelling tasks; nonetheless Laplace smoothing is transparent, easy to implement and sufficient to demonstrate the principle that drives this work.

D. The Viterbi Algorithm and Numerical Stability

Given the model parameters, the most probable tag sequence is found exactly by the Viterbi algorithm of Viterbi [3], a dynamic program that computes, for each position and each candidate tag, the score of the best path ending in that tag; its application to HMM decoding is described by Rabiner [1] and by Jurafsky and Martin [2]. The naive implementation multiplies probabilities, and since these are bounded above by one the cumulative product shrinks geometrically with sentence length and underflows to zero in double-precision arithmetic. The standard remedy, adopted here, is to perform the recursion on the logarithms of the probabilities, converting products into sums that never underflow.

III. HIDDEN MARKOV MODEL FORMULATION

Let the tagger operate over a set of N hidden states (POS tags) $S = \{s_1, \dots, s_N\}$ and a vocabulary of observable words V . A first-order HMM is defined by three sets of parameters. The initial distribution π gives the probability $\pi(s_i)$ that a sentence begins with tag s_i . The transition matrix A gives the

probability $P(s_j | s_i)$ of moving from tag s_i to tag s_j . The emission matrix B gives the probability $P(o_i | s_i)$ of observing word o_i while in tag s_i .

Given an observed word sequence $O = o_1 \dots o_T$, the tagging problem is to find the hidden tag sequence $Q = q_1 \dots q_T$ that maximises the joint probability

$$Q^* = \operatorname{argmax}_Q \pi(q_1) \cdot \prod_i P(q_i | q_{i-1}) \cdot P(o_i | q_i), \quad (1)$$

which factorises in exactly this way because of the first-order Markov and output-independence assumptions. Equation (1) is solved exactly by the Viterbi algorithm of Section IV-D. Fig. 1 illustrates this decoding as a trellis for a short example sentence, showing how the algorithm advances layer by layer and selects the single highest-scoring tag sequence.

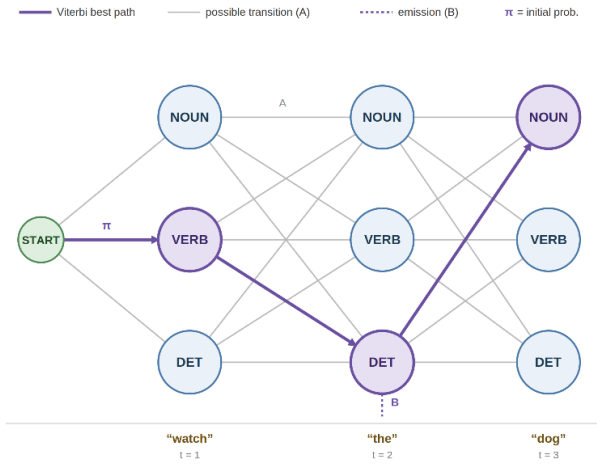


Fig. 1. HMM Architecture.

IV. METHODOLOGY

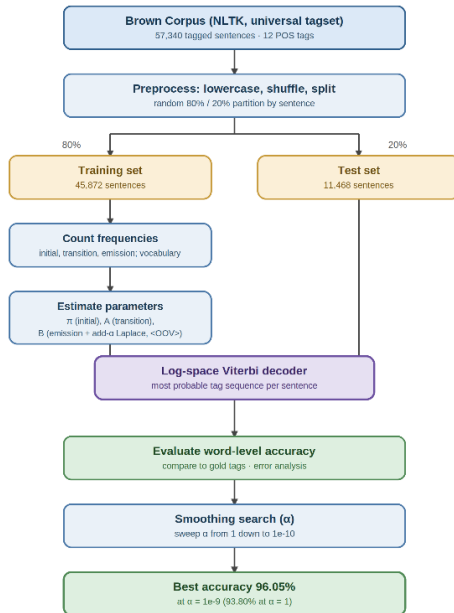


Fig. 2. Overview of the HMM POS tagging pipeline.

A. The Viterbi Algorithm and Numerical Stability

Fig. 2 summarises the complete tagging pipeline described in this section. The model is trained and evaluated on the Brown Corpus accessed through the Natural Language Toolkit (NLTK) with the universal tagset. The corpus contains 1,161,192 word tokens organised into 57,340 tagged sentences across fifteen genres, and the universal tagset reduces the annotation to twelve coarse categories. The sentences were randomly shuffled and then partitioned into an 80% training set and a 20% test set, giving 45,872 training sentences and 11,468 test sentences. Shuffling before splitting mixes the fifteen genres across both partitions, so that the training and test sets are drawn from the same distribution; this is preferable to a sequential split, which would group whole genres into one partition and bias the evaluation. All parameters of the final model are estimated from the training partition only, and the test partition is held out entirely for evaluation. To select the smoothing strength α without ever consulting the test data, a development split is taken inside the training partition: 10% of the training sentences (4,587 sentences) are reserved as a validation set and the remaining 41,285 sentences are used to fit the candidate models during the grid search of Section VI-A. Words are lower-cased before counting so that capitalised and lower-case occurrences of the same word share statistics. The shuffle uses a fixed random seed, so the partition and every figure reported below are exactly reproducible. Counting over the training partition yields a vocabulary of 45,153 distinct word types.

TABLE I. BROWN CORPUS STATISTICS (UNIVERSAL TAGSET)

Property	Value
Total word tokens	1,161,192
Total tagged sentences	57,340
Universal POS tags	12
Training sentences (80%)	45,872
Grid-search train sentences (90% of train)	41,285
Validation sentences (10% of train)	4,587
Test sentences (20%)	11,468
Training vocabulary types	45,153
Test word tokens	232,177

TABLE II. THE TWELVE UNIVERSAL POS TAGS

Tag	Category
NOUN	Noun (common and proper)
VERB	Verb
ADJ	Adjective
ADV	Adverb
PRON	Pronoun
DET	Determiner / article
ADP	Adposition (preposition / postposition)
NUM	Numeral
CONJ	Conjunction
PRT	Particle
X	Other / foreign / unknown
.	Punctuation

B. Parameter Estimation by Maximum Likelihood

The three parameter sets are estimated from training-set frequency counts. The initial probability of a tag is the fraction of training sentences that begin with it,

$$\pi(s_i) = \text{Count}(\text{sentence begins with } s_i) / M, \quad (2)$$

where M is the number of training sentences. The transition probability is the relative frequency of tag pairs,

$$P(s_j|s_i) = \text{Count}(s_i, s_j) / \text{Count}(s_i), \quad (3)$$

and the raw (unsmoothed) emission probability is the relative frequency with which a tag emits a word. The emission estimate is the one that must be smoothed, because it is the only parameter that involves the open-ended vocabulary; the transition and initial distributions are defined over the small, closed set of twelve tags and never encounter an unseen event at test time.

C. Laplace Smoothing and Out-of-Vocabulary Words

To prevent the zero probabilities that unseen words would otherwise receive, the emission matrix is smoothed with the add- α (Laplace) estimator. With pseudo-count α the smoothed probability of emitting word o_t from tag s_i is:

$$P_{\text{smooth}}(o_t|s_i) = [\text{Count}(o_t, s_i) + \alpha] / [\text{Count}(s_i) + \alpha(|V| + 1)], \quad (4)$$

with $\alpha = 1.0$ and $|V|$ the size of the training vocabulary. Crucially, each tag is also given a dedicated unseen-word entry, keyed “<OOV>”, whose probability is $\alpha / (\text{Count}(s_i) + \alpha(|V| + 1))$. Any test word absent from the training vocabulary is mapped to this entry, so it receives a small but non-zero, tag-dependent emission probability rather than zero. Because the unseen-word mass varies by tag, tags that were estimated from fewer tokens assign relatively more probability to the unseen entry. The smoothing alone gives the decoder a weak preference among tags for OOV words, which the transition probabilities then sharpen.

The unseen-word symbol is treated as one additional vocabulary item, which is why the denominator uses $\alpha(|V| + 1)$ rather than $\alpha|V|$; this keeps the smoothed probabilities over the known vocabulary together with the dedicated unseen entry summing to one for each tag. The pseudo-count α is left as a tunable hyperparameter rather than fixed at the textbook default. Because the conventional $\alpha = 1$ (add-one smoothing) over-smooths a vocabulary of this size, Section VI-A selects α by a grid search over candidate values spanning 10^{-6} down to 10^{-12} on the held-out validation set, retaining the value that maximises validation accuracy for the single final evaluation on the test partition.

D. Log-Space Viterbi Decoder

The Viterbi algorithm finds the highest-scoring tag sequence by dynamic programming. Define $v_t(j)$ as the score of the best tag path that ends in tag s_j at position t . Implemented multiplicatively, $v_t(j)$ is a product of $2t$ probabilities and underflows to zero for ordinary sentence lengths. Taking logarithms turns the product into a sum and the recursion becomes:

$$\log v_t(j) = \max_i^N [\log v_{t-1}(i) + \log P(s_j|s_i)] + \log P(o_t|s_j), \quad (5)$$

with the back-pointer recording, for each (t, j) , the predecessor tag i that attained the maximum. Because $\log 0$ is undefined, a tiny constant $\varepsilon = 10^{-12}$ is added to every probability before its logarithm is taken, which avoids math-domain errors without materially affecting the scores. Decoding proceeds in three stages: initialisation at $t = 1$ using $\log \pi$ and the first word’s log-emission; a forward pass applying (5) for each subsequent position and tag; and back-tracking from the highest-scoring final tag along the stored back-pointers to recover the optimal tag sequence. At every position an unseen word is looked up against the “<OOV>” entry of each tag, so OOV words are decoded by the same recursion as known words.

V. EXPERIMENTAL SETUP

A. Implementation Environment

The tagger was implemented in Python. NLTK was used only to download the Brown Corpus and to provide the universal tagset; all model components, the count dictionaries, the initial, transition and smoothed-emission tables, and the log-space Viterbi decoder were written from scratch using standard library containers, with no external HMM or tagging library. Counts were accumulated in a single pass over the training sentences, and the emission table was built explicitly over the full vocabulary so that the smoothed probability of every word under every tag, together with the per-tag unseen-word entry, was available to the decoder.

B. Evaluation Metric

Performance is measured by word-level accuracy, the fraction of test tokens whose predicted tag equals the true tag:

$$\text{Accuracy} = (\text{correctly tagged words}) / (\text{total test words}). \quad (6)$$

Every sentence in the test partition is decoded independently with the log-space Viterbi algorithm, the predicted tags are compared position-by-position with the true tags, and the matching and total counts are accumulated across the whole test set. Sentences containing at least one tagging error were additionally retained for the error analysis of Section VI-C.

VI. RESULTS AND DISCUSSION

A. Effect of the Smoothing Strength (α)

Table III reports the validation accuracy for each candidate value of α . Within the searched range the effect is modest but systematic: accuracy rises as α shrinks, from 95.47% at $\alpha = 10^{-6}$ to a plateau of 95.8529% that is first reached at $\alpha = 10^{-9}$ and is held for every smaller value down to 10^{-12} . The grid search therefore selects $\alpha = 10^{-9}$, the largest value that attains this maximum validation accuracy, and uses it for the final model. The trend has the same origin as the well-known weakness of the textbook default: α is added to the count of every one of the 45,153 vocabulary entries under every tag, so a large α floods each tag’s denominator with pseudo-count on the order of its real token count, flattening the smoothed emission distribution toward uniform and erasing the lexical evidence that distinguishes tags for known words.

This is why the conventional $\alpha = 1$ over-smooths so severely, attaining only 93.80% on the test set, far below every value in the grid. Shrinking α reverses the distortion: the probabilities of seen word-tag pairs are restored to nearly their maximum-likelihood values while the dedicated unseen-word entry still keeps OOV words taggable. Once α is small enough, further reduction no longer changes any decision, which is why the validation accuracy plateaus rather than continuing to climb. The α selected for this corpus is thus many orders of magnitude below the textbook default, a direct consequence of the vocabulary being large relative to the per-tag token counts.

TABLE III. WORD-LEVEL VALIDATION ACCURACY VERSUS SMOOTHING STRENGTH α (GRID SEARCH)

Smoothing strength α	Validation accuracy
1	93.4885%
10^{-6}	95.4712%
10^{-7}	95.6834%
10^{-8}	95.8409%
10^{-9}	95.8529%
10^{-10}	95.8507%
10^{-11}	95.8496%
10^{-12}	95.8496%

B. Overall Accuracy

On the held-out 20% test partition the textbook add-one setting ($\alpha = 1$) attains a word-level accuracy of 93.80%, correctly labelling 217,777 of 232,177 word tokens. This baseline is consistent with the accuracy typically reported for a well-implemented bigram HMM tagger on the Brown Corpus and confirms that the from-scratch parameter estimation, smoothing and log-space decoding are all functioning correctly. It is, however, far from optimal for a vocabulary of this size. Selecting the smoothing strength α by grid search on the validation set, as reported in Table III, identifies $\alpha = 10^{-9}$ as the best value, and re-fitting the emission model on the full training partition with this α raises test accuracy to 96.05%, correctly labelling 223,015 of 232,177 tokens. Because α was chosen on the validation set without reference to the test partition, this 96.05% is an unbiased estimate of generalisation accuracy rather than a figure tuned on the evaluation data. Table IV summarises both test-set evaluations.

TABLE IV. WORD-LEVEL ACCURACY ON THE HELD-OUT TEST SET

Configuration	Tokens correct / total	Word-level accuracy
Add-one baseline ($\alpha = 1$)	217,777 / 232,177	93.80%
Selected smoothing ($\alpha = 10^{-9}$)	223,015 / 232,177	96.05%

C. Error Analysis

The error analysis below is carried out on the selected $\alpha = 10^{-9}$ configuration, the model whose 96.05% test accuracy is reported above. To understand its residual 3.95% error rate, the first several test sentences containing at least one mistake were examined. Table V lists representative individual word errors drawn from these sentences, giving the word, the tag

the model predicted, the correct tag, and the most likely cause. Two recurring patterns emerge.

TABLE V. REPRESENTATIVE TAGGING ERRORS

Word	Prediction	True	Likely cause
<i>non-partisan</i>	DET	ADJ	Unseen adjective (OOV)
<i>according</i>	ADV	ADP	Function-word ambiguity
<i>offer</i>	VERB	NOUN	Noun-verb ambiguity
<i>average</i>	NOUN	VERB	Noun-verb ambiguity
<i>depending</i>	VERB	ADP	Participle read as a verb
<i>as</i>	ADV	ADP	Function-word ambiguity

The largest single source of error is the out-of-vocabulary problem, even though such out-of-vocabulary words make up only a small portion of the test tokens. The unseen compound adjective *non-partisan*, for example, was tagged DET instead of ADJ. For such words the smoothed emission probability is nearly uniform across tags, so the decision falls almost entirely to the transition probabilities, which favour whichever tag is most likely in the local grammatical context rather than the true category of the word. Smoothing keeps these words taggable, but it cannot supply the lexical knowledge the model never acquired.

The second pattern reflects the structural assumptions of the model. In the sentence “It was a very tempting offer.” the noun *offer* was tagged VERB, because *offer* is locally ambiguous between a noun and a verb reading, and that choice can be settled only by context wider than the single preceding tag a first-order model sees. The same short-context limitation accounts for *average* (tagged NOUN rather than VERB), *as* (tagged ADV rather than ADP) and *according* (tagged ADV rather than ADP). In every case the model selected the tag that maximised a purely local bigram score, which is exactly the behaviour the first-order Markov assumption prescribes.

D. The First-Order Markov Assumption as a Ceiling

Taken together, the errors show that the residual inaccuracy is not a bug in the implementation but a consequence of the model class. The first-order Markov assumption restricts the tagger to one tag of left context, so any disambiguation that depends on a word three or four positions away, agreement across a relative clause, the scope of a coordinating conjunction, or a noun phrase headed several words earlier is invisible to the decoder. The output-independence assumption compounds this for unseen words, because the model has no sub-word or morphological features (such as a hyphen, a capitalised first letter, or a characteristic suffix) that might signal the category of an unseen token such as *non-partisan*. These two assumptions, which are precisely what make exact and efficient decoding possible, are also what cap the achievable accuracy of the model.

E. Limitations

The tagger is a bigram (first-order) HMM and therefore cannot model dependencies beyond the immediately preceding tag. Out-of-vocabulary words are handled only by a single shared unseen-word probability per tag, with no use of morphological or orthographic cues that would

substantially improve rare-word tagging. The smoothing strength α was selected on a validation set carved from the training data and the test partition was used only for the single final evaluation, so the reported 96.05% is an unbiased estimate; the grid search was nonetheless coarse, sweeping α only by powers of ten, and a finer search or a larger validation set might shift the selected value slightly. Finally, accuracy is reported only as an aggregate word-level figure; a per-tag confusion analysis would localise the errors more precisely. Each of these points is a natural direction for extension.

VII. CONCLUSION

This paper described the construction of a Hidden Markov Model part-of-speech tagger from first principles and its training and evaluation on the Brown Corpus with the twelve-tag universal tagset. The model parameters were estimated by maximum-likelihood counting, the emission matrix was smoothed with add- α Laplace smoothing and equipped with an explicit out-of-vocabulary probability so that unseen words remained taggable, and the Viterbi decoder was reformulated in log-space to eliminate the floating-point underflow that otherwise defeats long-sentence decoding. The textbook add-one setting was found to over-smooth a vocabulary of this size, reaching only 93.80% on the held-out test partition. Selecting the smoothing strength by grid search on a validation set drawn from the training data, rather than on the test set, identified $\alpha = 10^{-9}$ and raised word-level accuracy to an unbiased 96.05% on the test partition, correctly labelling 223,015 of 232,177 tokens.

The error analysis showed that the remaining errors stem from two sources that are intrinsic to the model rather than the implementation: unseen or rare content words, for which the smoothed emission carries too little information to override the local context, and grammatical dependencies that exceed the one-step memory of the first-order Markov assumption. These findings articulate both the value of the HMM transparent, exactly decodable and strong on common words and its ceiling. Future work will raise that ceiling by extending the model to a second-order (trigram) HMM, by adding morphological and orthographic features for unseen-word handling, and by comparing the HMM against a discriminative or neural sequence labeller on the same partition to quantify the gain available from relaxing the Markov and independence assumptions.

REFERENCES

- [1] L. R. Rabiner, “A tutorial on hidden Markov models and selected applications in speech recognition,” *Proc. IEEE*, vol. 77, no. 2, pp. 257–286, 1989.
- [2] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. (draft). Stanford University, 2023.
- [3] A. J. Viterbi, “Error bounds for convolutional codes and an asymptotically optimum decoding algorithm,” *IEEE Trans. Inf. Theory*, vol. 13, no. 2, pp. 260–269, 1967.
- [4] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. Sebastopol, CA: O’Reilly Media, 2009.
- [5] W. N. Francis and H. Kučera, *Brown Corpus Manual*. Providence, RI: Brown University, 1979.
- [6] S. Petrov, D. Das, and R. McDonald, “A universal part-of-speech tagset,” in *Proc. 8th Int. Conf. Language Resources and Evaluation (LREC)*, 2012, pp. 2089–2096.
- [7] K. W. Church, “A stochastic parts program and noun phrase parser for unrestricted text,” in *Proc. 2nd Conf. Applied Natural Language Processing*, 1988, pp. 136–143.
- [8] T. Brants, “TnT – a statistical part-of-speech tagger,” in *Proc. 6th Conf. Applied Natural Language Processing*, 2000, pp. 224–231.
- [9] C. D. Manning and H. Schütze, *Foundations of Statistical Natural Language Processing*. Cambridge, MA: MIT Press, 1999.
- [10] S. F. Chen and J. Goodman, “An empirical study of smoothing techniques for language modeling,” *Comput. Speech Lang.*, vol. 13, no. 4, pp. 359–394, 1999.